01-Classical Classifier Baseline

In this notebook, we will do a simple baseline classification using classicial method (Tfidf and logistic regression)

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import precision_recall_fscore_support, f1_score, classificati
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
```

In [2]:
```
# load train and dev set
train_fname = "../../data/pitt/pitt_train_txt.csv"
dev_fname = "../../data/pitt/pitt_dev_txt.csv"

train_df = pd.read_csv(train_fname)
dev_df = pd.read_csv(dev_fname)

print("Train len =", len(train_df))
print("Dev df =", len(dev_df))
```

```
Train len = 337
Dev df = 115
```

In [3]:
```
train_df.head()
```

Out[3]:

	id	file	target	text	split_name
0	3	003-0.cha	AD	here's a cookie jar .\nand the lid is off the ...	train
1	5	005-0.cha	AD	the boy is slipping off the stool .\nhe's tryi...	train
2	5	005-2.cha	AD	okay ‡ he's fallin(g) off a chair .\nshe's &-...	train
3	7	007-1.cha	AD	xxx .\ncan I look at it and tell you ?\noh oka...	train
4	7	007-3.cha	AD	well the girl is telling the boy to get the co...	train

In [4]:
```
dev_df.head()
```

Out[4]:

	id	file	target	text	split_name
0	16	016-0.cha	AD	guy or a boy reaching for a cookie jar &-uh ti...	dev
1	16	016-1.cha	AD	uh the water's spilling over .\ndrying the dis...	dev
2	16	016-3.cha	AD	a boy is reaching for the cookie jar and tripp...	dev
3	16	016-4.cha	AD	well the little girl wants the cookie that the...	dev
4	33	033-0.cha	AD	okay .\napparently a mother and a daughter and...	dev

Pre-processing

We will lowercase the text for better efficiency

In [5]:
```
train_df["text_lower"] = train_df["text"].apply(lambda text: text.lower().strip())
dev_df["text_lower"] = dev_df["text"].apply(lambda text: text.lower().strip())
```

In [6]:
```
train_df.head()
```

Out[6]:

	id	file	target	text	split_name	text_lower
0	3	003-0.cha	AD	here's a cookie jar .\nand the lid is off the ...	train	here's a cookie jar .\nand the lid is off the ...
1	5	005-0.cha	AD	the boy is slipping off the stool .\nhe's tryi...	train	the boy is slipping off the stool .\nhe's tryi...
2	5	005-2.cha	AD	okay ‡ he's fallin(g) off a chair .\nshe's &-...	train	okay ‡ he's fallin(g) off a chair .\nshe's &-...
3	7	007-1.cha	AD	xxx .\ncan I look at it and tell you ?\noh oka...	train	xxx .\ncan i look at it and tell you ?\noh oka...
4	7	007-3.cha	AD	well the girl is telling the boy to get the co...	train	well the girl is telling the boy to get the co...

In [7]:
```
dev_df.head()
```

Out[7]:

	id	file	target	text	split_name	text_lower
0	16	016-0.cha	AD	guy or a boy reaching for a cookie jar &-uh ti...	dev	guy or a boy reaching for a cookie jar &-uh ti...
1	16	016-1.cha	AD	uh the water's spilling over .\ndrying the dis...	dev	uh the water's spilling over .\ndrying the dis...
2	16	016-3.cha	AD	a boy is reaching for the cookie jar and tripp...	dev	a boy is reaching for the cookie jar and tripp...
3	16	016-4.cha	AD	well the little girl wants the cookie that the...	dev	well the little girl wants the cookie that the...
4	33	033-0.cha	AD	okay .\napparently a mother and a daughter and...	dev	okay .\napparently a mother and a daughter and...

Build Tfidf Vectorizer

We can use Tfidf Vectorizer to convert text to a fixed-length vector

In [8]:
```
vectorizer = TfidfVectorizer()
```

```
corpus = train_df["text_lower"]
print(len(corpus))
print("Building Tfidf Vectorizer...")
X_train = vectorizer.fit_transform(corpus)
print("Completed")
```

```
337
Building Tfidf Vectorizer...
Completed
```

In [9]:
```
print(X_train.shape)
```

```
(337, 1437)
```

Train Logistic regression using Tfidf vectorizer

We can now train a simple logistic regression classifier using processed X_train

In [10]:
```
X_train = vectorizer.fit_transform(train_df["text_lower"])
y_train = train_df["target"]

print("Training logistic classifier...")
clf = LogisticRegression(random_state=42,
                         class_weight="balanced",
                        ).fit(X_train, y_train)
print("Completed")
```

```
Training logistic classifier...
Completed
```

In [11]:
```
# getting score on train set
train_score = clf.score(X_train, y_train)
print(f"Train accuracy = {train_score*100:.2f} %")
```

```
Train accuracy = 87.54 %
```

Get accuracy using cross validation

In [12]:
```
print("Performing cross-validation...")
scores = cross_val_score(clf, X_train, y_train, cv=5)
print("Completed")
print(scores)
print("Average =", np.average(scores))
```

```
Performing cross-validation...
Completed
[0.64705882 0.75       0.70149254 0.74626866 0.80597015]
Average = 0.7301580333625988
```

In [13]:
```
# get precision / recall / f1
y_pred = clf.predict(X_train)
print(classification_report(y_train, y_pred, target_names=["AD", "CN"]))
```

```
                precision    recall  f1-score   support

          AD       0.91      0.86      0.88       187
          CN       0.84      0.89      0.86       150

    accuracy                           0.88       337
   macro avg       0.87      0.88      0.87       337
weighted avg       0.88      0.88      0.88       337
```

Now we can evaluate on dev set

In [14]:
```
X_dev = vectorizer.transform(dev_df["text_lower"])
y_dev = dev_df["target"]
print("X_dev shape =", X_dev.shape)
print("y_dev len =", len(y_dev))
```

```
X_dev shape = (115, 1437)
y_dev len = 115
```

In [15]:
```
dev_score = clf.score(X_dev, y_dev)
print(f"Dev accuracy = {dev_score*100:.2f} %")
```

```
Dev accuracy = 74.78 %
```

In [16]:
```
y_pred = clf.predict(X_dev)
print(classification_report(y_dev, y_pred, target_names=["AD", "CN"]))
```

```
              precision    recall  f1-score   support

          AD       0.81      0.72      0.76        64
          CN       0.69      0.78      0.73        51

    accuracy                           0.75       115
   macro avg       0.75      0.75      0.75       115
weighted avg       0.75      0.75      0.75       115
```

In [17]:
```
### Process prediction for test set
```

In [18]:
```
test_fname = "../../data/pitt/pitt_test_txt.csv"

test_df = pd.read_csv(test_fname)
print("Test df =", len(test_df))
test_df["text_lower"] = test_df["text"].apply(lambda text: text.lower().strip())
test_df.head()
```

```
Test df = 97
```

Out[18]:

	id	file	target	text	split_name	text_lower
0	1	001-0.cha	AD	mhm .\nalright .\nthere's &-um a young boy tha...	test	mhm .\nalright .\nthere's &-um a young boy tha...
1	1	001-2.cha	AD	mhm .\nthere's a young boy &-uh going in a coo...	test	mhm .\nthere's a young boy &-uh going in a coo...
2	23	023-0.cha	AD	(..) well I see the mother .\nwhat's she doin(...	test	(..) well i see the mother .\nwhat's she doin(...
3	23	023-2.cha	AD	well you see <the little> the little boy stea...	test	well you see <the little> the little boy stea...
4	39	039-0.cha	AD	okay we see &-um first of all &-uh the sink is...	test	okay we see &-um first of all &-uh the sink is...

In [19]:
```
X_test = vectorizer.transform(test_df["text_lower"])
test_df["prediction"] = clf.predict(X_test)
test_df.head()
```

Out[19]:

	id	file	target	text	split_name	text_lower	prediction
0	1	001-0.cha	AD	mhm .\nalright .\nthere's &-um a young boy tha...	test	mhm .\nalright .\nthere's &-um a young boy tha...	CN
1	1	001-2.cha	AD	mhm .\nthere's a young boy &-uh going in a coo...	test	mhm .\nthere's a young boy &-uh going in a coo...	AD
2	23	023-0.cha	AD	(..) well I see the mother .\nwhat's she doin(...	test	(..) well i see the mother .\nwhat's she doin(...	AD
3	23	023-2.cha	AD	well you see <the little> the little boy stea...	test	well you see <the little> the little boy stea...	AD
4	39	039-0.cha	AD	okay we see &-um first of all &-uh the sink is...	test	okay we see &-um first of all &-uh the sink is...	AD

In [20]:
```
correct_count = len(test_df.query("target == prediction"))
print(f"Correct count = {correct_count}, accuracy = {correct_count*100/len(test_df)
```

```
Correct count = 82, accuracy = 85 %
```

In [21]:
```
print(classification_report(test_df["target"], test_df["prediction"]))
```

	precision	recall	f1-score	support
AD	0.87	0.85	0.86	55
CN	0.81	0.83	0.82	42
accuracy			0.85	97
macro avg	0.84	0.84	0.84	97
weighted avg	0.85	0.85	0.85	97